

Testing and Debugging

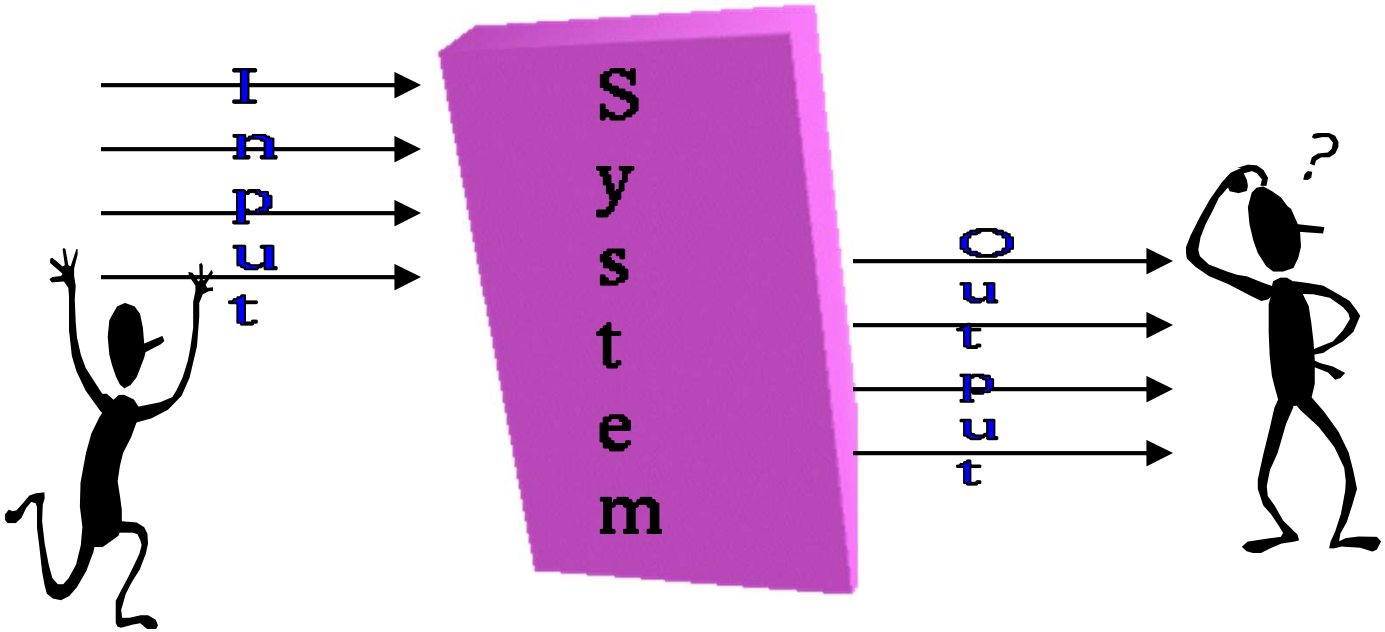
Organization of this lecture

- Important concepts in program testing
- Black-box testing:
  - equivalence partitioning
  - boundary value analysis
- White-box testing
- Debugging
- Unit, Integration, and System testing
- Summary

How do you test a program?

- Input test data to the program.
- Observe the output:
  - Check if the program behaved as expected.

How do you test a system?



How do you test a system?

- If the program does not behave as expected:
  - note the conditions under which it failed.
  - later debug and correct.

Error, Faults, and Failures

- A failure is a manifestation of an error (aka defect or bug).
  - mere presence of an error may not lead to a failure.

Error, Faults, and Failures

- A fault is an incorrect state entered during program execution:
  - a variable value is different from what it should be.
  - A fault may or may not not lead to a failure.

Example

- Imagine the hospital system calculates the appointment fee.
- The correct fee is: **Consultation = ₹500**
- But the programmer accidentally writes: fee = 50;
- **Error/Bug:** The programmer wrote 50 instead of 500.
- **Fault:** During execution, the variable fee contains 50 instead of 500.
- **Failure:** The patient sees:
- Consultation Fee: ₹50
- when it should be ₹500.

- **Error → Fault → Failure**
- **Mistake in code → Wrong internal state → Wrong result seen by user**

Test cases and Test suites

- Test a software using a set of carefully designed test cases:
  - the set of all test cases is called the test suite
    - TC1 → Valid appointment
    - TC2 → Already booked slot
    - TC3 → Empty doctor field
    - TC4 → Invalid date
    - TC5 → Invalid patient details

Test cases and Test suites

- A test case is a triplet [I,S,O]
  - I is the data to be input to the system,
  - S is the state of the system at which the data will be input,
  - O is the expected output of the system.

- **Hospital example**
- Suppose we test appointment booking:
- **Test Case = [I, S, O]**
- **I:** Patient selects Dr. Anil at 10:00 AM
- **S:** 10:00 AM slot is available
- **O:** Appointment should be booked successfully
- Another:
- **I:** Patient selects Dr. Anil at 10:00 AM
- **S:** 10:00 AM slot is already booked
- **O:** System should display "Slot unavailable"

Verification versus Validation

- Verification is the process of determining:
  - whether output of one phase of development conforms to its previous phase.
- Validation is the process of determining
  - whether a fully developed system conforms to its SRS document.

Verification versus Validation

- Verification is concerned with phase containment of errors,
  - whereas the aim of validation is that the final product be error free.

Design of Test Cases

- Exhaustive testing of any non-trivial system is impractical:
  - input data domain is extremely large.
- Design an optimal test suite:
  - of reasonable size and
  - uncovers as many errors as possible.

Design of Test Cases

- If test cases are selected randomly:
  - many test cases would not contribute to the significance of the test suite,
  - would not detect errors not already being detected by other test cases in the suite.
- Number of test cases in a randomly selected test suite:
  - not an indication of effectiveness of testing.

Design of Test Cases

- Testing a system using a large number of randomly selected test cases:
  - does not mean that many errors in the system will be uncovered.
- Consider an example for finding the maximum of two integers x and y.

Design of Test Cases

- The code has a simple programming error:
- If (x>y) max = x;
  else max = x;
- test suite {(x=3,y=2);(x=2,y=3)} can detect the error,
- a larger test suite {(x=3,y=2);(x=4,y=3); (x=5,y=1)} does not detect the error.

Design of Test Cases

- Systematic approaches are required to design an optimal test suite:
  - each test case in the suite should detect different errors.

Design of Test Cases

- There are essentially two main approaches to design test cases:
  - Black-box approach
  - White-box (or glass-box) approach

Black-box Testing

- Test cases are designed using only functional specification of the software:
  - without any knowledge of the internal structure of the software.
- For this reason, black-box testing is also known as functional testing.

White-box Testing

- Designing white-box test cases:
  - requires knowledge about the internal structure of software.
  - white-box testing is also called structural testing.

Black-box Testing

- There are essentially two main approaches to design black box test cases:
  - Equivalence class partitioning
  - Boundary value analysis

Equivalence Class Partitioning

- Input values to a program are partitioned into equivalence classes.
- Partitioning is done such that:

  divide the inputs into groups where the system is expected to behave similarly.

Why define equivalence classes?

- Test the code with just one representative value from each equivalence class:
  - as good as testing using any other values from the equivalence classes.

Equivalence Class Partitioning

- How do you determine the equivalence classes?
  - examine the input data.
  - few general guidelines for determining the equivalence classes can be given

Equivalence Class Partitioning

- If the input data to the program is specified by a range of values:
  - e.g. numbers between 1 to 5000.
  - one valid and two invalid equivalence classes are defined.

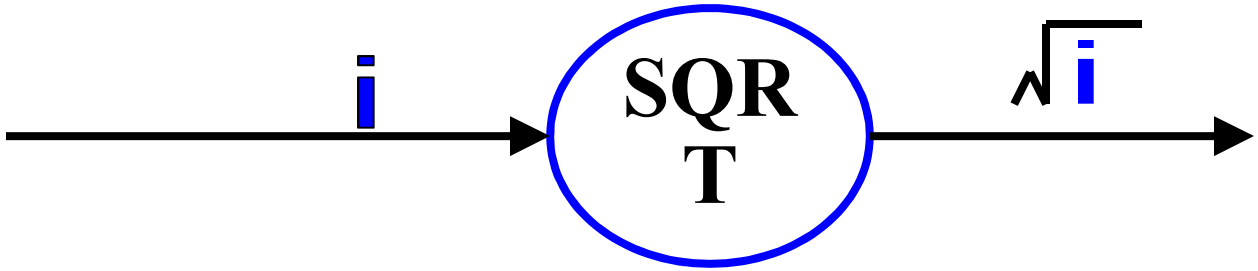


Equivalence Class Partitioning

- If input is an enumerated set of values:
  - e.g. {a,b,c}
  - one equivalence class for valid input values
  - another equivalence class for invalid input values should be defined.

Example

- A program reads an input value in the range of 1 and 5000:
  - computes the square root of the input number



Example (cont.)

- There are three equivalence classes:
  - the set of negative integers,
  - set of integers in the range of 1 and 5000,
  - integers larger than 5000.



Example (cont.)

- The test suite must include:
  - representatives from each of the three equivalence classes:
  - a possible test suite can be: {-5,500,6000}.



Boundary Value Analysis

- Some typical programming errors occur:
  - at boundaries of equivalence classes
  - might be purely due to psychological factors.
- Programmers often fail to see:
  - special processing required at the boundaries of equivalence classes.

Boundary Value Analysis

- Programmers may improperly use < instead of <=
- Boundary value analysis:
  - select test cases at the boundaries of different equivalence classes.

- Example:
- Valid marks: 0 to 100
- Test values around the boundaries:
- -1, 0, 1 99, 100, 101

- Can write if (marks > 0 && marks < 100)
- instead of:
- if (marks >= 0 && marks <= 100)
- The mistake becomes visible when we test **0 and 100**.

Example

- For a function that computes the square root of an integer in the range of 1 and 5000:
  - test cases must include the values: {0,1,5000,5001}.



Debugging

- Once errors are identified:
  - it is necessary identify the precise location of the errors and to fix them.
- Each debugging approach has its own advantages and disadvantages:
  - each is useful in appropriate circumstances.

Brute-force method

- This is the most common method of debugging:
  - least efficient method.
  - program is loaded with print statements
  - print the intermediate values
  - hope that some of printed values will help identify the error.

Symbolic Debugger

- Brute force approach becomes more systematic:
  - with the use of a symbolic debugger,
  - symbolic debuggers get their name for historical reasons
  - early debuggers let you only see values from a program dump:
    - determine which variable it corresponds to.

Symbolic Debugger

- Using a symbolic debugger:
  - values of different variables can be easily checked and modified
  - single stepping to execute one instruction at a time
  - break points and watch points can be set to test the values of variables.

Backtracking

- This is a fairly common approach.
- Beginning at the statement where an error symptom has been observed:
  - source code is traced backwards until the error is discovered.

Backtracking

- Unfortunately, as the number of source lines to be traced back increases,
  - the number of potential backward paths increases
  - becomes unmanageably large for complex programs.

Cause-elimination method

- Determine a list of causes:
  - which could possibly have contributed to the error symptom.
  - tests are conducted to eliminate each.
- A related technique of identifying error by examining error symptoms:
  - software fault tree analysis.

Program Slicing

- This technique is similar to back tracking.
- However, the search space is reduced by defining slices.
- A slice is defined for a particular variable at a particular statement:
  - set of source lines preceding this statement which can influence the value of the variable.

Debugging Guidelines

- Debugging usually requires a thorough understanding of the program design.
- Debugging may sometimes require full redesign of the system.
- A common mistake novice programmers often make:
  - not fixing the error but the error symptoms.

Debugging Guidelines

- Be aware of the possibility:
  - an error correction may introduce new errors.
- After every round of error-fixing:
  - regression testing must be carried out.

Regression testing

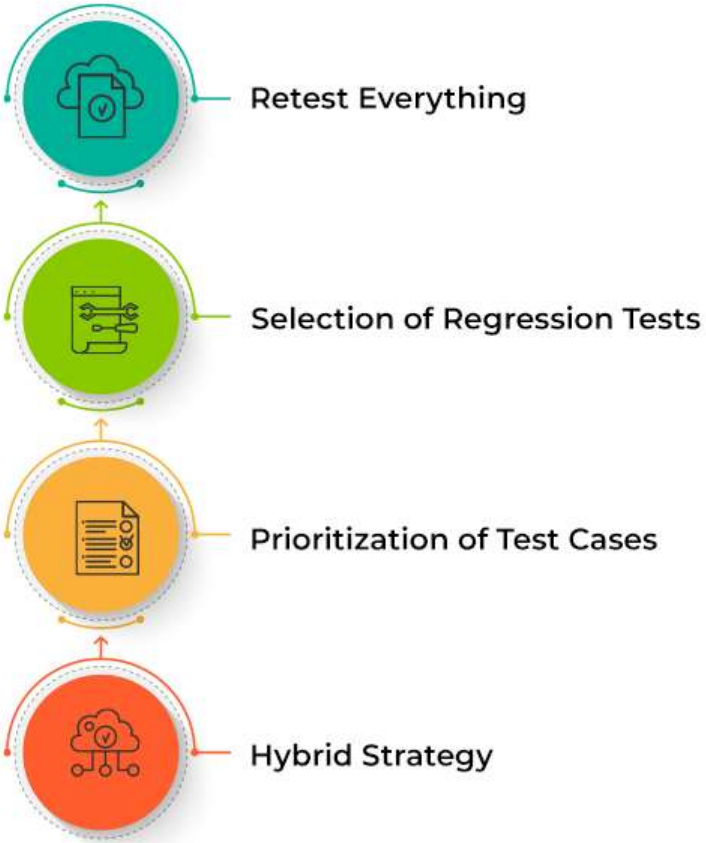
- ***Regression testing is defined as a type of software testing technique that re-runs functional and non-functional tests to ensure that a software application works as intended after any code changes, updates, revisions, improvements, or optimizations. This article explains regression testing in detail, its fundamental techniques, and the top five regression testing tools.***

REGRESSION TESTING

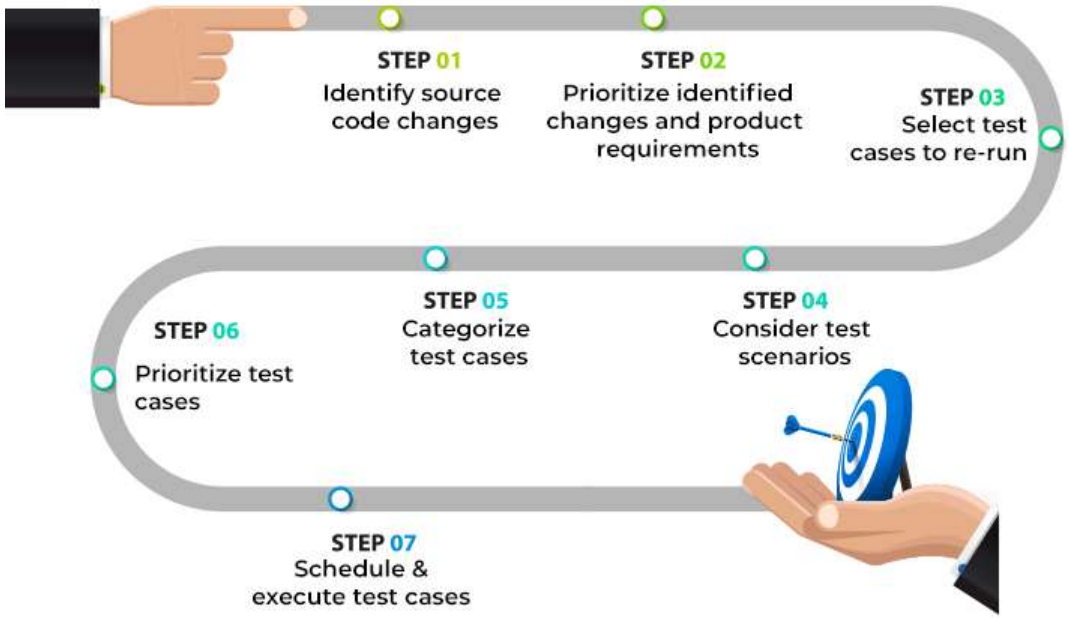
- Each time a new module is added as part of integration testing
  - New data flow paths are established
  - New I/O may occur
  - New control logic is invoked
- Due to these changes, may cause problems with functions that previously worked flawlessly.
- *Regression testing is the* ***re-execution*** of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.
- Whenever software is corrected, some aspect of the software configuration (the program, its documentation, or the data that support it) is changed.

- Regression testing is the activity that helps to ensure that changes do not introduce unintended behavior or additional errors.
- **Regression testing may be conducted manually, by re-executing a subset of all test cases or using automated *capture/playback tools.***
- Capture/playback tools enable the software engineer to capture test cases and results for subsequent playback and comparison.
- Regression testing contains 3 diff. classes of test cases:
  - **A representative sample of tests that will exercise all software functions**
  - **Tests that focus on the software components that have been changed.**
  - **Additional tests that focus on software functions that are likely to be affected by the change.**

REGRESSION TESTING TECHNIQUES



HOW TO PERFORM REGRESSION TESTING?



Advantages of Regression Testing

- Integration risk is minimized
- The quality of the end product is improved.
- Error diagnosis and correction are simplified.
- Progress is easier to assess.



- As integration testing proceeds, the number of regression tests can grow quite large.
- It is impractical and inefficient to re-execute every test for every program function once a change has occurred.

Smoke testing

- Smoke testing is an testing approach that is commonly used when product software is developed.
- It determines whether the deployed software build is stable or not.
- It allows the software team to assess the project on a frequent basis.
- A smoke test checks if the software is stable enough for further testing, while a regression test checks if existing features still work correctly after changes or updates.

Activities of smoke testing

- Software components that have been translated into code are integrated into a *build*.
  - *A build includes*
    - *data files,*
    - *libraries,*
    - *reusable modules,*
    - engineered components that are required to implement one or more product functions.
- A series of tests is designed to expose errors that will keep the build from properly performing its function.
  - The intent should be to uncover "show-stopper" errors
- The build is integrated with other builds, and the entire product is smoke tested daily.

Smoke Testing vs. Regression Testing

Criteria	Smoke Testing	Regression Testing
Purpose	To ensure that the critical functionalities of the application are working fine after new changes	To ensure that changes do not break existing functionality
Testing scope	It tests only critical functionalities of the application	It tests the entire application or specific areas that are affected by changes
Test type	It is a quick and less time-consuming testing approach	It is a comprehensive and time-consuming testing approach
Test result analysis	It identifies critical issues that could prevent the release of a new build or version	It compares the current test results with previous test results to identify any regression issues
Test result reporting	It generates a summary report on the critical functionalities tested and their status	It generates detailed reports on the tests performed and any issues found

Program Analysis Tools

- An automated tool:
  - takes program source code as input
  - produces reports regarding several important characteristics of the program,
  - such as size, complexity, adequacy of commenting, adherence to programming standards, etc.

Program Analysis Tools

- Some program analysis tools:
  - produce reports regarding the adequacy of the test cases.
- There are essentially two categories of program analysis tools:
  - Static analysis tools
  - Dynamic analysis tools

Static Analysis Tools

- Static analysis tools:
  - assess properties of a program without executing it.
  - Analyze the source code
    - provide analytical conclusions.

Static Analysis Tools

- Whether coding standards have been adhered to?
  - Commenting is adequate?
- Programming errors such as:
  - uninitialized variables
  - mismatch between actual and formal parameters.
  - Variables declared but never used, etc.

Static Analysis Tools

- Code walk through and inspection can also be considered as static analysis methods:
  - however, the term static program analysis is generally used for automated analysis tools.

Dynamic Analysis Tools

- Dynamic program analysis tools require the program to be executed:
  - its behavior recorded.
  - Produce reports such as adequacy of test cases.

Testing

- The aim of testing is to identify all defects in a software product.
- However, in practice even after thorough testing:
  - one cannot guarantee that the software is error-free.

Testing

- The input data domain of most software products is very large:
  - it is not practical to test the software exhaustively with each input data value.

Testing

- Testing does however expose many errors:
  - testing provides a practical way of reducing defects in a system
  - increases the users' confidence in a developed system.

Testing

- Testing is an important development phase:
  - requires the maximum effort among all development phases.
- In a typical development organization:
  - maximum number of software engineers can be found to be engaged in testing activities.

Testing

- Many engineers have the wrong impression:
  - testing is a secondary activity
  - it is intellectually not as stimulating as the other development activities, etc.

Testing

- Testing a software product is in fact:
  - as much challenging as initial development activities such as specification, design, and coding.
- Also, testing involves a lot of creative thinking.

Testing

- Software products are tested at three levels:
  - Unit testing
  - Integration testing
  - System testing

Unit testing

- During unit testing, modules are tested in isolation:
  - If all modules were to be tested together:
    - it may not be easy to determine which module has the error.

Unit testing

- Unit testing reduces debugging effort several folds.
  - Programmers carry out unit testing immediately after they complete the coding of a module.

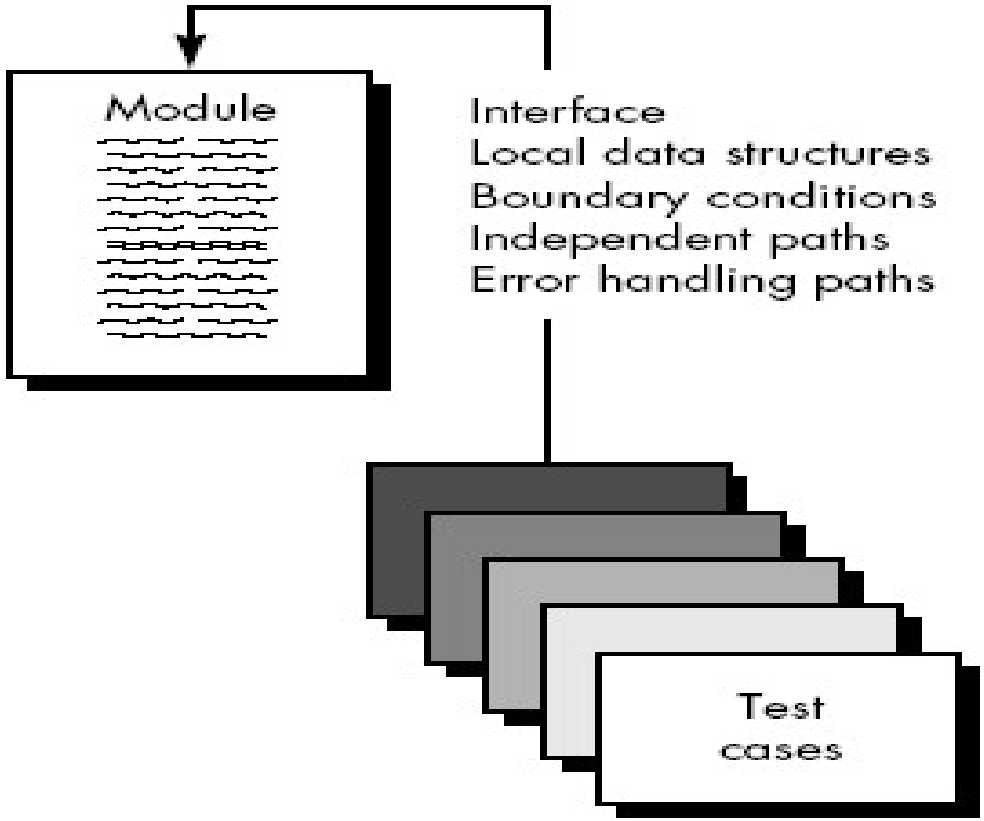
UNIT TESTING

- Focuses verification effort on the smallest unit of software design – component or module.
- Using the component-level design description as a guide
  - important control paths are tested to uncover errors within the boundary of the module.
- Unit test is white-box oriented, and the step can be conducted in parallel for multiple components.
- Unit test consists of
  - Unit Test Considerations
  - Unit Test Procedures

Tests involved in unit testing

- The module interface is tested
  - to ensure that information properly flows into and out of the program unit under test.
- Local data structures are examined
  - to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution.
- All independent paths through the control structure are exercised
  - to ensure that all statements in a module have been executed at least once.
- Boundary conditions are tested
  - to ensure that the module operates properly at boundaries established to limit or restrict processing.
- And finally, all error-handling paths are tested.

Unit Test Considerations



- Selective testing of *execution paths* is an essential task during the unit test.
- Test cases should be designed to uncover errors due to
  - Computations,
  - Incorrect comparisons, or
  - Improper control flow

- *Basis path* and loop testing are effective techniques for uncovering a broad array of *path errors*.

Errors are commonly found during unit testing

- More common errors in computation are
  - misunderstood or incorrect arithmetic precedence
  - mixed mode operations,
  - incorrect initialization,
  - precision inaccuracy,
  - incorrect symbolic representation of an expression.
- Comparison and control flow are closely coupled to one another
  - Comparison of different data types,
  - Incorrect logical operators or precedence,
  - Incorrect comparison of variables
  - Improper or nonexistent loop termination,
  - Failure to exit when divergent iteration is encountered
  - improperly modified loop variables.

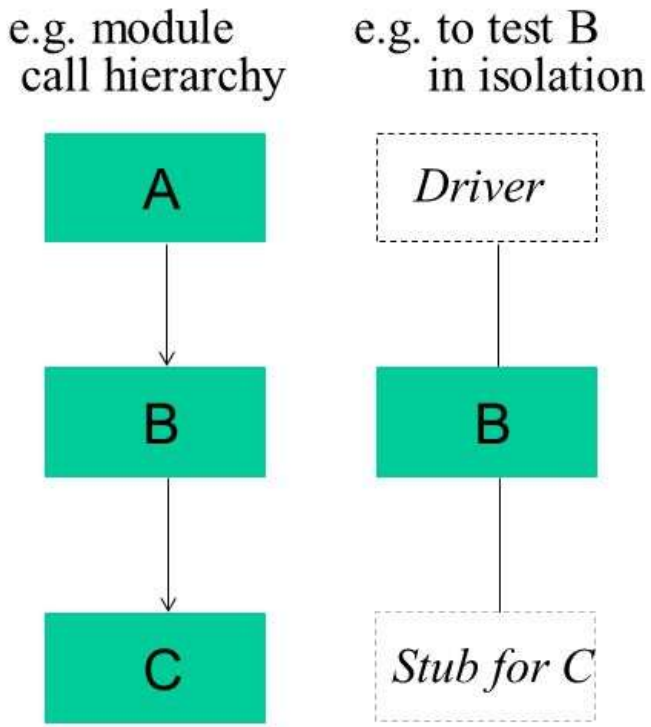
- Software often fails at its boundaries. That is, errors often occur when the *n*th element of an *n*-dimensional array is processed or when the maximum or minimum allowable value is encountered.

- So BVA test is always be a last task for unit test.

- Test cases that exercise data structure, control flow, and data values just below, at, and just above maxima and minima are very likely to uncover errors.

What are Stubs, Drivers ?

- Stub
  - **dummy module** which simulates the functionality of a module called by a given module under test
- Driver
  - a module which transmits test cases in the form of input arguments to the given module under test and either prints or interprets the results produced by it



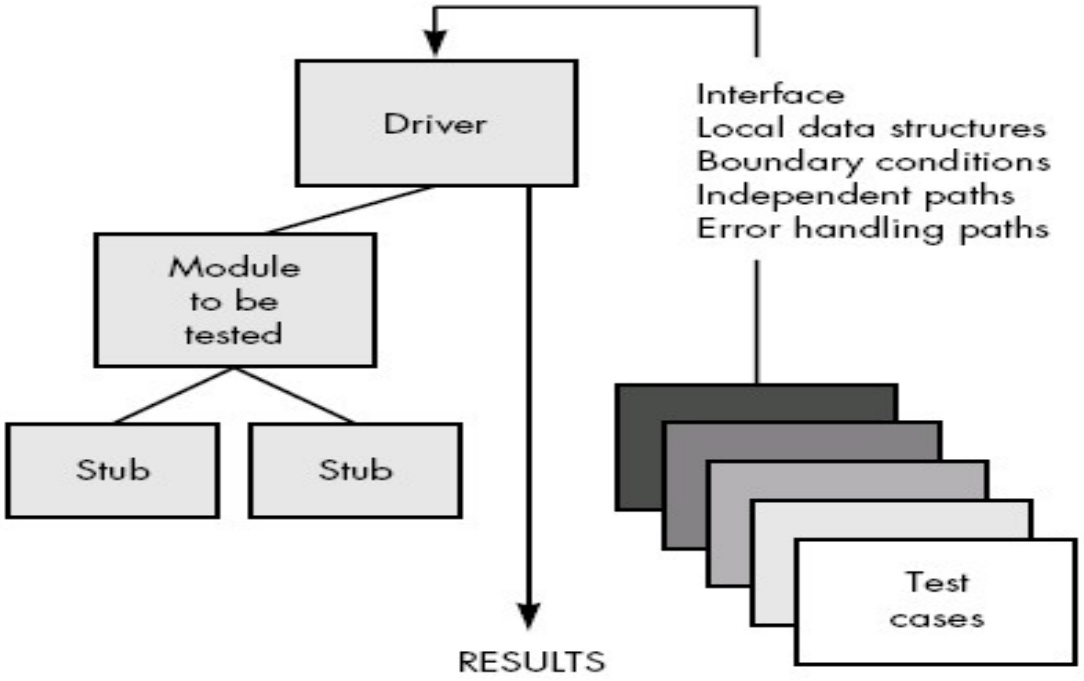
Unit Test Procedures

- Perform before coding or after source code has been generated.
- A review of design information provides guidance for establishing test cases. Each test case should be coupled with a set of expected results.
- Because a component is not a stand-alone program, driver and/or stub software must be developed for each unit test.

Unit Test Procedures

- In most applications a driver is nothing more than a "main program" that accepts test case data, passes such data to the component (to be tested), and prints relevant results.
- A stub or "dummy subprogram" uses the subordinate module's interface, may do minimal data manipulation, prints verification of entry, and returns control to the module undergoing testing.
- *Stubs* serve to replace modules that are subordinate the component to be tested.

Unit Test Procedures



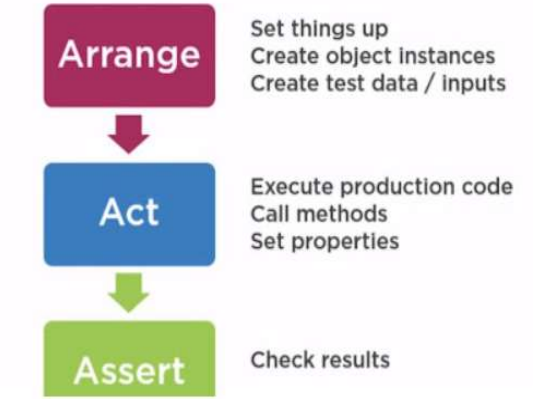
Unit Test Environment

- Drivers and stubs represent overhead. That is, both are software that must be written but that is not delivered with the final software product.

- In such cases, complete testing can be postponed until the integration test step.

- When only one function is addressed by a component, the number of test cases is reduced and errors can be more easily predicted and uncovered.

- 3A rule



Program Unit

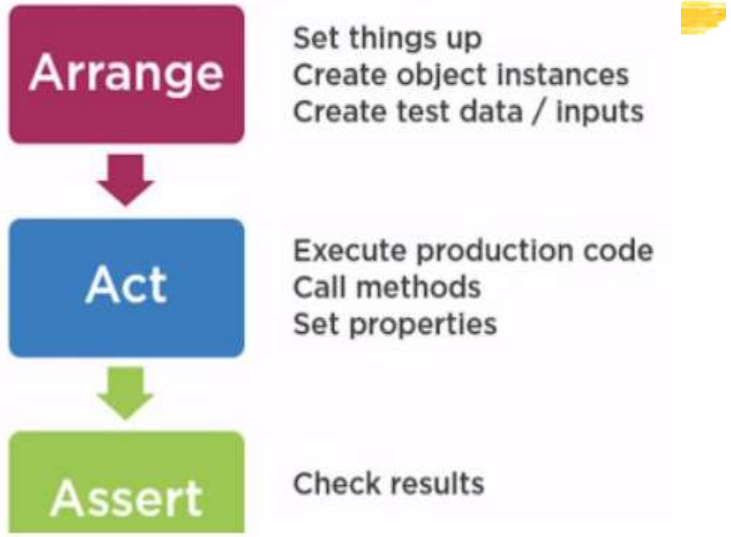
```
namespace Calculator.Library
{
    public class Calculator
    {
        public static int Divide(int numerator, int denominator)
        {
            int result = numerator / denominator;
            return result;
        }
    }
}
```

```using System;
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace Calculator.Test
{
    [TestClass]
    public class CalculatorTest
    {
        [TestMethod]
        public void Test_Divide()
        {
            // Arrange
            int expected = 5;
            int numerator = 20;
            int denominator = 4;

            // Act
            int actual = Calculator.Library.Calculator.Divide(numerator, denominator);

            // Assert
            Assert.AreEqual(expected, actual);
        }
    }
```





- Problems may arise when the modules are put together
- *Problems caused*
  - Data can be lost across an interface
  - one component can have an adverse effect on another
  - sub functions, when combined, may not produce the desired major function
  - Global data structures can present problems.

  This lead to the need of integration testing, even after unit testing

Integration testing

- After different modules of a system have been coded and unit tested:
  - modules are integrated in steps according to an integration plan
  - partially integrated system is tested at each integration step.

Integration Testing

- Develop the integration plan by examining the structure chart :
  - big bang approach
  - top-down approach
  - bottom-up approach
  - mixed approach

Big bang Integration Testing

- Big bang approach is the simplest integration testing approach:
  - all the modules are simply put together and tested.
  - this technique is used only for very small systems.

Big bang Integration Testing

- Main problems with this approach:
  - if an error is found:
    - it is very difficult to localize the error
    - the error may potentially belong to any of the modules being integrated.
  - debugging errors found during big bang integration testing are very expensive to fix.

Top-down integration testing

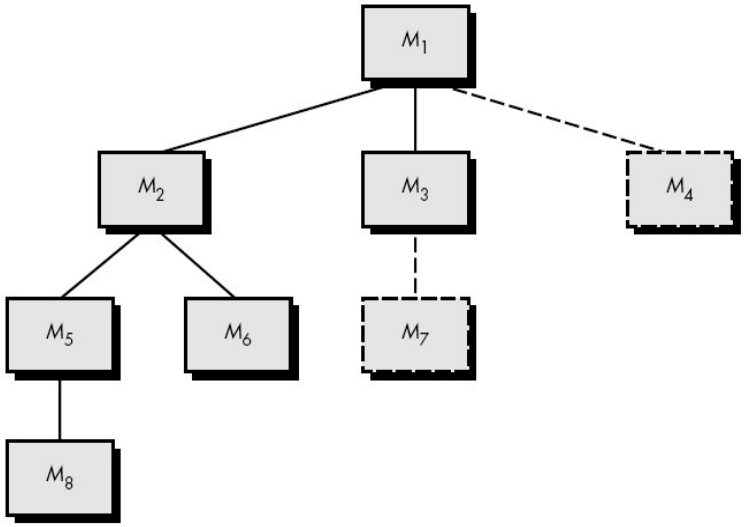
- Top-down integration testing starts with the main routine:
  - and one or two subordinate routines in the system.
- After the top-level 'skeleton' has been tested:
  - immediate subordinate modules of the 'skeleton' are combined with it and tested.

TOP-DOWN INTEGRATION TESTING

- It is an incremental approach to construct the software architecture.
- Modules are integrated by moving downward through the control hierarchy,
- Integration begins with the main control module (main program).
- Modules that are subordinate and ultimately subordinate to the main control module are incorporated into the structure by
  - Depth-first
  - Breadth-first manner.

Depth First Integration

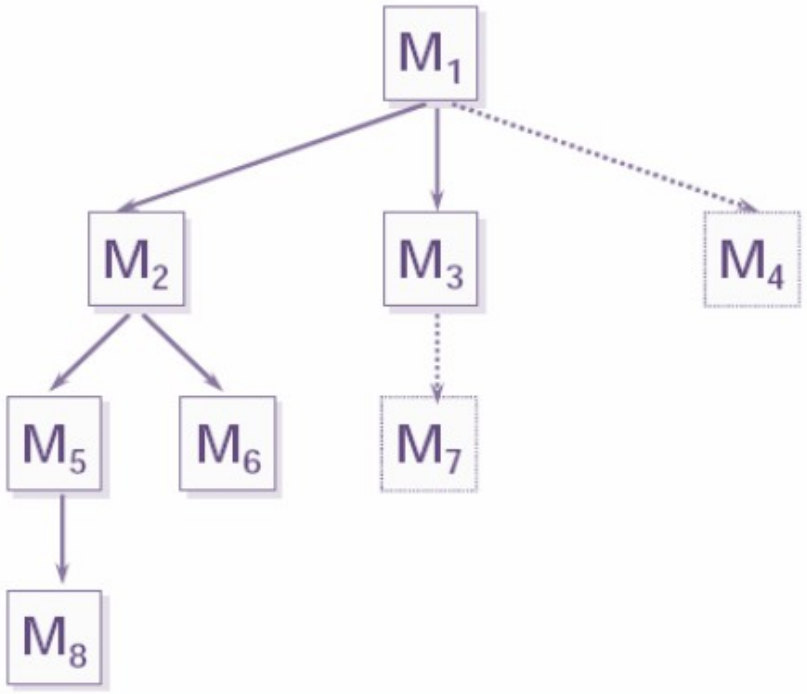
- It integrates all components on a major control path of the program structure.
- Selection of a major path depends on application-specific characteristics.
- Eg“ if we select left path,
  - components M1, M2 , M5 would be integrated first.
  - Next, M8 or M6 would be integrated.
  - Then, the central and right-hand control paths are built.



Breadth first integration

- Breadth-first integration incorporates all components directly subordinate at each level, moving across the structure horizontally.
- Step would be:
  - components M2, M3, and M4 would be integrated first
  - next control level, M5, M6, and so on follows.

- **Top-Down Testing with Depth-First**
  - M1, M2, M5, M8
  - M6
  - M3, M7
  - M4
- **Top-Down Testing with Breath-First**
  - M1
  - M2, M3, M4
  - M5, M6, M7
  - M8



Top-down Integration process five steps:

1. The main control module is used as a test driver and stubs are substituted for all components directly subordinate to the main control module.
2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated
4. On completion of each set of tests, another stub is replaced with the real component.
5. The process continues from step 2 until the entire program structure is built. Regression testing may be conducted to ensure that new errors have not been introduced.

Problem occur in top-down integration

- Logic related problems can arise
- Most common problems occurs when processing at low levels in the hierarchy is required to adequately test upper levels.
- **No significant data can flow upward** in the program structure due to stubs replace low level modules at the beginning of top-down testing. In this case, Tester will have 3 choice
  - Delay many tests until stubs are replaced with actual modules
  - develop stubs that perform limited functions that simulate the actual module
  - integrate the software from the bottom of the hierarchy upward

E-commerce Application

Hierarchy Levels:

- **Lower Level**: Inventory Management System
- **Middle Level**: Order Processing System
- **Upper Level**: User Interface (UI) on the e-commerce website

Scenario:

- The inventory management system stub does not properly handle negative stock quantities. If a product goes out of stock and the inventory system allows for a negative stock value, this incorrect data gets passed to the order processing system.
- **Consequence**: When customers attempt to purchase items, the order processing system may erroneously confirm orders for out-of-stock products, leading to canceled orders and customer dissatisfaction at the UI level.

Bottom-up Integration Testing

- Integrate and test the bottom level modules first.
- A disadvantage of bottom-up testing:
  - when the system is made up of a large number of small subsystems.
  - This extreme case corresponds to the big bang approach.

Bottom-up Integration

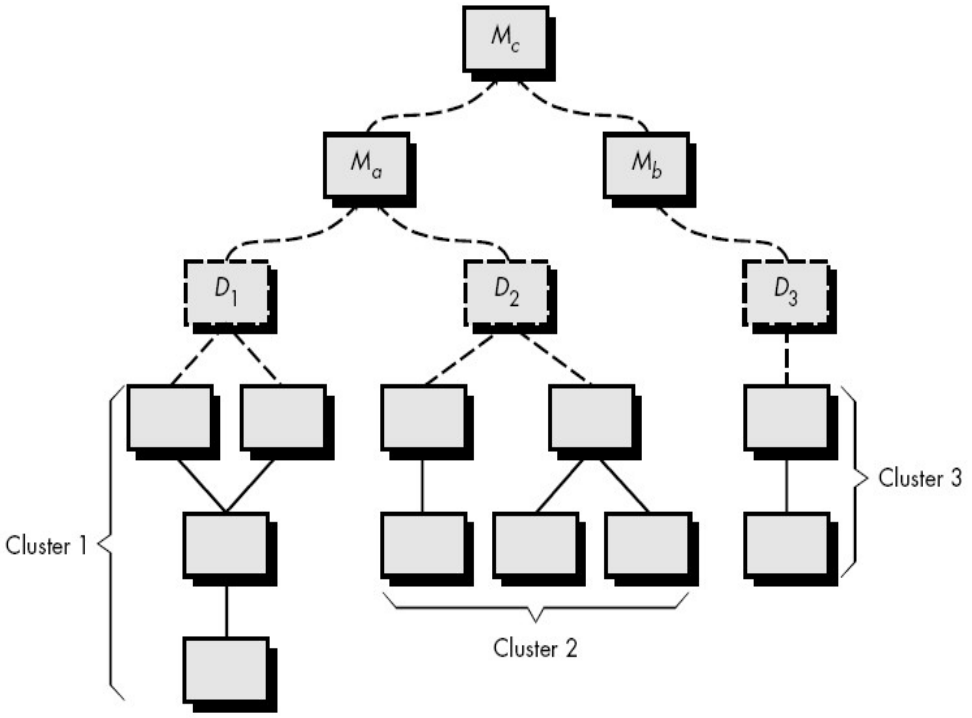
- Bottom-up integration testing, as its name implies, begins construction and testing with atomic modules (i.e., components at the lowest levels in the program structure)
- Because components are integrated from the bottom up, processing required for components subordinate to a given level is always available and the need for stubs is eliminated.

Bottom up integration process steps

- Low-level components are combined into clusters (sometimes called *builds)* that perform a specific software sub function.
- A driver (a control program for testing) is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure.

Example

- Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver.
- Components in clusters 1 and 2 are subordinate to Ma.
- Drivers D1 and D2 are removed and the clusters are interfaced directly to Ma. Similarly, driver D3 for cluster 3 is removed prior to integration with module Mb.
- Both Ma and Mb will ultimately be integrated with component Mc, and so forth.



Strategic options Of Integration Testing

- Selection of integration strategy depends on

1. Software characteristics

- The top-down approach is particularly effective in scenarios where a clear understanding of the overall system architecture and user experience is essential.
- The bottom-up approach is particularly effective in scenarios where detailed implementation and technical aspects of a system need to be prioritized and where smaller components can be built and tested independently.

2. Project schedule

Time Constraints:

If a project has tight deadlines, teams may choose integration strategies that allow for faster feedback and earlier delivery of functional components.

Strategy: Bottom-Up Integration

Testing and Validation Requirement:

Projects that necessitate thorough testing and validation of components before full integration may require a strategy that emphasizes early and frequent testing.

Strategy: Top-Down Integration

Strategic options Of Integration Testing

- Couples top down & bottom up strategies
  - Uses top down tests for upper levels of program
  - Bottom up tests are applied on subordinates

- When integration testing is conducted, tester must identify critical modules
- Characteristics of critical modules
  - Addresses several software requirements
  - Has high level of control
  - Complex or error prone
  - Has definite performance requirements
  - They should be tested early
  - Regression test must focus on critical module functions

Integration test documentation

- An overall plan for integration of the software and a description of specific tests is documented in a Test Specification.
- Testing is divided into phases
- Following tests are applied to these phases
  - Interface integrity
  - Functional validity
  - Information content
  - performance
- This incorporates
  - a test plan- overall strategy of integration
  - and a test procedure- order of integration

- Interface integrity
  - Internal & external interfaces are tested as each module is incorporated into the structure
- Functional validity
  - Uncover the functional errors
- Information content
  - Tests designed to uncover errors associated with local or global data structures
- Performance
  - Tests designed to verify performance bounds established during software design



- Test plan describes following
  - A schedule for integration
  - The development of overhead software
  - Start and end dates for each phase
  - Test environment and resources are
  - Unusual hardware configurations
  - Exotic simulators
  - Special test tools or techniques

Contents of test procedure

- The order of integration
- corresponding tests at each integration step are described.
- A listing of all test cases
- expected results are also included.

Mixed integration testing

- Mixed (or sandwiched) integration testing:
  - uses both top-down and bottom-up testing approaches.
  - Most common approach

Integration Testing

- In top-down approach:
  - testing waits till all top-level modules are coded and unit tested.
- In bottom-up approach:
  - testing can start only after bottom level modules are ready.

System Testing

- System testing involves:
  - validating a fully developed system against its requirements.

SYSTEM TESTING

- System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system.
- Although each test has a different purpose, all work to verify that system elements have been properly integrated and perform allocated functions.
- Types of system tests are:
  - Recovery Testing
  - Security Testing
  - Stress Testing

Recovery Testing

- Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed.
- If recovery is automatic (performed by the system itself), reinitialization, check pointing mechanisms, data recovery, and restart are evaluated for correctness.
- If recovery requires human intervention, that is mean-time-to-repair (MTTR) is evaluated to determine whether it is within acceptable limits.

Banking application that allows users to transfer funds between accounts. Recovery testing in this scenario would involve simulating a failure during the fund transfer process (e.g., a network crash, power outage, or server failure) and then verifying that the system can recover.

1. Test Setup: A user initiates a fund transfer of $1,000 from Account A to Account B.
2. Failure Simulation: Halfway through the transaction, the network connection is intentionally severed, or the system crashes.
3. Recovery Testing Objective:

After the failure, the system should be able to restore its previous state when restarted.

The recovery process should ensure data integrity, such as checking that no funds were deducted from Account A, or if they were, they were successfully credited to Account B.

The user should be notified of the failure and be provided with the status of the transfer.

Expected Outcome:

Upon recovery, the system should correctly resume the transaction from where it left off, or it should roll back to its previous consistent state (i.e., no partial transfer should occur).

The banking application should log the failure and provide the user with appropriate feedback, such as "Transaction Failed" or "Please Retry."

This ensures that the banking system can handle unexpected failures without data loss or corruption, maintaining user trust.

Security Testing

- *Security testing* attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration.
- During security testing, the tester plays the role(s) of the individual who desires to break through the system.
- Given enough time and resources, good security testing will ultimately penetrate a system.
- The role of the system designer is to make penetration cost more than the value of the information that will be obtained.
- The tester may attempt to acquire passwords through externally, may attack the system with custom software designed to breakdown any defenses that have been constructed; may browse through insecure data; may purposely cause system errors.

Stress Testing

- *Stress testing* executes a system in a manner that demands resources in abnormal quantity, frequency, or volume.

For example,

1. special tests may be designed that generate ten interrupts per second
2. Input data rates may be increased by an order of magnitude to determine how input functions will respond
3. test cases that require maximum memory or other resources are executed
4. test cases that may cause excessive hunting for disk-resident data are created

- A variation of stress testing is a technique called *sensitivity testing*

Aspect	Stress Testing	Sensitivity Testing
Definition	Testing done to evaluate the system's behavior under **extreme or beyond-normal load conditions**.	Testing done to observe how **small changes in input or parameters** affect the system's performance or output.
Purpose	To identify the **breaking point** and ensure system recovers gracefully after failure.	To determine **which factors most influence performance or stability**.
Focus Area	System **robustness**, **stability**, and **recovery** under heavy load.	System **responsiveness** and **tuning** under minor variations.
Key Metrics Observed	Response time, throughput, error rate, recovery time.	Performance variation, latency, accuracy, resource utilization.
Example (Amazon)	Simulating 20 million users during a sale when normal load is 5 million — to check system stability.	Increasing database latency by 5% to observe effect on checkout time — to identify sensitivity.
Outcome	Ensures system can handle overload and recover gracefully.	Helps optimize system by focusing on most impactful parameters.
Testing Stage	Performed before deployment or release.	Performed during **performance optimization** or **tuning** phase.
Failure Handling	System may crash — recovery behavior is analyzed.	System should not crash — only performance fluctuation observed.

VALIDATION TESTING

- Validation testing succeeds when software functions in a manner that can be reasonably expected by the customer.
- Like all other testing steps, validation tries to uncover errors, but the focus is at the requirements level— on things that will be immediately apparent to the end-user.
- Reasonable expectations are defined in the Software Requirements Specification— a document that describes all user-visible attributes of the software.
- Validation testing comprises of
  - **Validation Test criteria**
  - **Configuration review**

Validation Test criteria

- It is achieved through a series of tests that demonstrate agreement with requirements.
- A *test plan* outlines the classes of *tests to be conducted*
- A *test procedure* defines specific test cases that will be used to demonstrate *agreement with requirements*.
- Both the plan and procedure are designed to ensure that
  - all functional requirements are satisfied,
  - all behavioral characteristics are achieved,
  - all performance requirements are attained,
  - documentation is correct,
  - other requirements are met

Validation Test criteria

- After each validation test case has been conducted, one of two possible conditions exist:

1. The function or performance characteristics conform to specification and are accepted
2. A deviation from specification is uncovered and a deficiency list is created

Configuration Review

- The intent of the review is to ensure that all elements of the software configuration have been *properly developed*, *are cataloged*, and have the necessary detail to the support phase of the software life cycle.
- The configuration review, sometimes called an *audit*.

- Validation test succeeds when
  - software functions in a manner that can be reasonably expected by the customer
- Reasonable expectations are defined in SRS document
  - It describes all user visible attributes of the software
  - SRS has a section called validation criteria
    - Information in this section forms the basis of validation testing

USER ACCEPTANCE TESTING

What is UAT?

User Acceptance Testing (UAT) is a type of testing performed by the end user or the client to verify/accept the software system before moving the software application to the production environment. UAT is done in the final phase of testing after functional, integration and system testing is done.

Purpose of UAT

The main **Purpose of UAT** is to validate end to end business flow. It does not focus on cosmetic errors, spelling mistakes or system testing. User Acceptance Testing is carried out in a separate testing environment with production-like data setup. It is kind of black box testing where two or more end-users will be involved.

Alpha and Beta Testing

- When custom software is built for one customer, a series of *acceptance tests* are conducted to enable the customer to validate all requirements.

- Conducted by the end-user rather than software engineers, an acceptance test can range from an informal "test drive" to a planned and systematically executed series of tests.

- Most software product builders use a process called alpha and beta testing to uncover errors that only the end-user seems able to find.

Alpha Testing

Alpha Testing is a type of acceptance testing; performed to identify all possible issues and bugs before releasing the final product to the end users. Alpha testing is carried out by the testers who are internal employees of the organization. The main goal is to identify the tasks that a typical user might perform and test them.

To put it as simple as possible, this kind of testing is called alpha only because it is done early on, near the end of the development of the software, and before beta testing. The main focus of alpha testing is to simulate real users by using a black box and white box techniques.



Beta Testing

Beta Testing is performed by "real users" of the software application in "real environment" and it can be considered as a form of external User Acceptance Testing. It is the final test before shipping a product to the customers. Direct feedback from customers is a major advantage of Beta Testing. This testing helps to test products in customer's environment.

Beta version of the software is released to a limited number of end-users of the product to obtain feedback on the product quality. Beta testing reduces product failure risks and provides increased quality of the product through customer validation.

Alpha Testing	Beta Testing
Alpha testing performed by Testers who are usually internal employees of the organization	Beta testing is performed by Clients or End Users who are not employees of the organization
Alpha Testing performed at developer's site	Beta testing is performed at a client location or end user of the product
Reliability and Security Testing are not performed in-depth Alpha Testing	Reliability, Security, Robustness are checked during Beta Testing
Alpha testing involves both the white box and black box techniques	Beta Testing typically uses Black Box Testing
Alpha testing requires a lab environment or testing environment	Beta testing doesn't require any lab environment or testing environment. The software is made available to the public and is said to be real time environment
Long execution cycle may be required for Alpha testing	Only a few weeks of execution are required for Beta testing
Critical issues or fixes can be addressed by developers immediately in Alpha testing	Most of the issues or feedback is collected from Beta testing will be implemented in future versions of the product

Types of Beta Testing

There are different types of Beta tests in software testing, and they are as follows:

Traditional Beta testing: Product is distributed to the target market, and related data is gathered in all aspects. This data can be used for Product improvement.

Public Beta Testing: Product is publicly released to the outside world via online channels and data can be gathered from anyone. Based on feedback, product improvements can be done. For example, Microsoft conducted the largest of all Beta Tests for its OS -- Windows 8 before officially releasing it.

Technical Beta Testing: Product is released to the internal group of an organization and gathers feedback/data from the employees of the organization.

Focused Beta: Product is released to the market for gathering feedback on specific features of the program. For example, important functionality of the software.

Phases of Testing

Alpha and Beta tests are typically carried for "off-the-shelf" software or product-oriented companies. The Phases of Testing for a product company typically varies from a service-oriented organization. Following is the testing phase adopted by product firms



Pre-Alpha:- Software is a prototype. UI is complete. But not all features are completed. At this stage, software is not published.

Alpha: Software is near its development and is internally tested for bugs/issues

Beta: Software is stable and is released to a limited user base. The goal is to get customer feedback on the product and make changes in software accordingly

Release Candidate (RC): Based on the feedback of Beta Test, you make changes to the software and want to test out the bug fixes. At this stage, you do not want to make radical changes in functionality but just check for bugs. RC is also put out to the public

How many errors are still remaining?

- Seed the code with some known errors:
  - artificial errors are introduced into the program.
  - Check how many of the seeded errors are detected during testing.

Error Seeding

- Let:
  - N be the total number of errors in the system
  - n of these errors be found by testing.
  - S be the total number of seeded errors,
  - s of the seeded errors be found during testing.

Error Seeding

- n/N = s/S
- N = S n/s
- remaining defects:

  N - n = n ((S - s)/ s)

Example

- 100 errors were introduced.
- 90 of these errors were found during testing
- 50 other errors were also found.
- Remaining errors=
  50 (100-90)/90 = 6

Error Seeding

- The kind of seeded errors should match closely with existing errors:
  - However, it is difficult to predict the types of errors that exist.
- Categories of remaining errors:
  - can be estimated by analyzing historical data from similar projects.

Summary

- Exhaustive testing of almost any non-trivial system is impractical.
  - we need to design an optimal test suite that would expose as many errors as possible.

Summary

- If we select test cases randomly:
  - many of the test cases may not add to the significance of the test suite.
- There are two approaches to testing:
  - black-box testing
  - white-box testing.

Summary

- Black box testing is also known as functional testing.
- Designing black box test cases:
  - requires understanding only SRS document
  - does not require any knowledge about design and code.
- Designing white box testing requires knowledge about design and code.

Summary

- We discussed black-box test case design strategies:
  - equivalence partitioning
  - boundary value analysis
- We discussed some important issues in integration and system testing.